

LAPORAN MILESTONE 4 TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA
Intermediate Code & Interpreter



Disusun oleh:

Juan Oloando Simanungkalit 13524032

Edward David Rumahorbo 13524036

Reinhard Alfonzo Hutabarat 13524056

Manuel Timothy Silalahi 13524102

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL.GANESA 10, BANDUNG 40132
2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB 1	
LANDASAN TEORI.....	1
1. Intermediate Code Generation.....	1
2. Three-Address Code (TAC).....	1
3. Translasi Control Flow.....	2
4. Decorated Abstract Syntax Tree (DAST).....	3
5. Interpreter dan Virtual Machine.....	4
6. Stack Machine dan Memori Eksekusi.....	5
7. Daftar Instruksi TAC Arion.....	6
8. Instruksi OPR dan Operasi Stack.....	7
9. Pengelolaan Scope Leksikal dan Static Link.....	8
BAB 2	
PERANCANGAN DAN IMPLEMENTASI.....	12
1. Perancangan.....	12
1.1. Arsitektur Umum.....	12
1.2. Struktur Program.....	13
1.3 Alur Program.....	14
2. Implementasi.....	15
2.1. Kakas yang Digunakan.....	15
2.2. Kelas CodeBuffer dan Struktur Instruksi.....	15
2.3. Kelas CodeGenerator.....	16
2.4. Kelas RuntimeValue.....	24
2.5. Kelas Interpreter.....	24
2.6. Kelas RuntimeErrorManager.....	29
2.7. Kelas DecoratedASTReader.....	30
2.8. Alur Kerja Main.cpp.....	31
2.9. Implementasi Lanjutan (selain spesifikasi).....	32
BAB 3	
PENGUJIAN.....	33
1. Test Case 1.....	33
2. Test Case 2.....	35
3. Test Case 3.....	38
4. Test Case 4.....	41
5. Test Case 5.....	44
BAB 4	
KESIMPULAN DAN SARAN.....	53

1. Kesimpulan.....	53
2. Saran.....	53
LAMPIRAN.....	55
REFERENSI.....	55

BAB 1

LANDASAN TEORI

1. *Intermediate Code Generation*

Intermediate Code Generation merupakan tahap pertama dalam fase *back-end* sebuah compiler atau interpreter. Setelah fase *front-end* (yang mencakup analisis leksikal, sintaksis, dan semantik) berhasil menghasilkan *Decorated Abstract Syntax Tree* (DAST) yang telah tervalidasi sepenuhnya, tahap ini bertugas mengubah representasi pohon yang bersifat hierarkis tersebut menjadi serangkaian instruksi yang bersifat linier dan sekuensial. Rangkaian instruksi linier inilah yang disebut sebagai *Intermediate Code*.

Tujuan utama dari pembuatan *Intermediate Code* adalah menjembatani ketidaksesuaian antara cara berpikir manusia, yang alami dalam bentuk pohon bersarang, dengan cara kerja mesin yang beroperasi secara linier, instruksi demi instruksi. Memori komputer dan CPU dirancang untuk mengeksekusi instruksi secara berurutan, sehingga traversal langsung terhadap *parse tree* yang kompleks bukan merupakan pendekatan yang efisien.

Pemisahan arsitektur antara *front-end* dan *back-end* melalui *Intermediate Code* memberikan keuntungan modularitas yang signifikan. Jika di masa mendatang sintaks bahasa Arion diperbarui, hanya bagian *front-end* saja yang perlu diubah, sementara mesin eksekusi *back-end* tetap dapat bekerja tanpa modifikasi, selama *Intermediate Code* yang dihasilkan memiliki format yang sama.

Algoritma yang digunakan untuk menghasilkan *Intermediate Code* dari DAST adalah pendekatan berbasis *Depth-First Search* (DFS). Traversal dilakukan dari akar pohon menuju daun, mengevaluasi sub-ekspresi terdalam terlebih dahulu sebelum menyusun instruksi untuk ekspresi induknya. Pendekatan ini memastikan bahwa setiap nilai yang dibutuhkan oleh sebuah operasi sudah tersedia di dalam memori pada saat operasi tersebut dieksekusi.

2. *Three-Address Code (TAC)*

Three-Address Code (TAC) adalah format representasi *Intermediate Code* yang paling banyak digunakan dalam industri compiler modern. Dinamakan *Three-Address* karena setiap baris instruksi dirancang sesederhana mungkin, di mana setiap instruksi hanya boleh melibatkan maksimal tiga alamat memori, yaitu dua operand sumber dan satu lokasi tujuan. Struktur dasar sebuah instruksi TAC adalah sebagai berikut:

```
hasil := operand1 operator operand2
```

Dengan membatasi setiap instruksi hanya pada tiga alamat, TAC memaksa pemecahan ekspresi-ekspresi yang panjang dan kompleks menjadi langkah-langkah atomik yang mudah dieksekusi oleh mesin virtual. Proses pemecahan ini memanfaatkan *variabel sementara* (biasanya dilambangkan sebagai *t1*, *t2*, *t3*, dan seterusnya) yang berperan sebagai tempat penyimpanan hasil antara.

Sebagai contoh, ekspresi $x := (a + b) * (c - d)$ yang memiliki empat operan dalam pohon bersarang, akan diratakan menjadi instruksi TAC berikut:

```
t1 := a + b
t2 := c - d
t3 := t1 * t2
x  := t3
```

Pada implementasi Arion Interpreter Milestone 4, format instruksi TAC yang digunakan diadaptasi dengan konvensi khusus. Setiap instruksi memiliki format: [nomor_baris] [instruksi] [level_hierarki] [operand]. Misalnya, instruksi 1 LIT 0 10 berarti: pada baris ke-1, muat nilai literal 10 dari level hierarki 0 ke dalam stack.

3. *Translasi Control Flow*

Mengubah ekspresi matematika menjadi TAC relatif lugas karena polanya bersifat deterministik. Tantangan sesungguhnya muncul saat meratakan struktur *control flow*, yaitu konstruksi percabangan (*if-else*) dan perulangan (*while*). Kedua konstruksi ini secara inheren bersifat bersarang (*nested*) dalam pohon, namun harus direpresentasikan secara linier dalam TAC.

Solusi standar untuk masalah ini adalah menggunakan kombinasi Label dan Jump (GOTO). Label adalah penanda statis yang merepresentasikan posisi baris tertentu dalam TAC, sedangkan instruksi Jump memindahkan *Instruction Pointer* (IP) ke posisi label yang dituju. Terdapat dua jenis Jump:

- **Jump Bersyarat (JPC - Jump if Condition False):** melompat ke suatu label hanya jika nilai di puncak stack bernilai 0 (false). Instruksi ini digunakan untuk merepresentasikan percabangan if-else dan kondisi while.
- **Jump Mutlak (JMP - Unconditional Jump):** melompat ke suatu label tanpa syarat apapun. Instruksi ini digunakan untuk melompati blok else setelah blok then selesai, atau untuk kembali ke awal kondisi while setelah body selesai dieksekusi.

Penggunaan *JPC (jump if false)* dipilih karena memanfaatkan konsep *fall-through*, ketika kondisi bernilai *true*, eksekusi dapat langsung berlanjut ke instruksi berikutnya tanpa perlu lompatan tambahan, sehingga menyederhanakan alur kontrol. Pada Milestone 4 ini, penggunaan *JPC* bersifat wajib.

Contoh translasi pernyataan *if $x > 5$ then $y := x * 2$ else $y := x \text{ div } 2$* ke dalam TAC menggunakan konvensi instruksi Arion:

```

LOD 0 [addr_x] ; muat nilai x

LIT 0 5 ; muat literal 5

OPR 0 11 ; GTR:  $x > 5$ 

JPC 0 [baris_else] ; jika false, loncat ke else

... (instruksi then) ...

JMP 0 [baris_end] ; loncat ke akhir if-else

[baris_else]: ... (instruksi else) ...

[baris_end]: ...

```

4. *Decorated Abstract Syntax Tree (DAST)*

Decorated Abstract Syntax Tree (DAST) adalah *Abstract Syntax Tree (AST)* yang telah diperkaya atau didekorasi dengan informasi semantik sebagai hasil dari proses analisis semantik. DAST merupakan sumber kebenaran tunggal yang digunakan oleh *Intermediate Code Generator* pada Milestone 4.

Keberadaan DAST berstatus wajib bagi *Intermediate Code Generator* karena generator tidak dapat bekerja hanya berdasarkan struktur teks sumber. Sebagai contoh, ketika

generator menemui ekspresi $a + b$ dalam pohon, tanpa informasi tipe, generator tidak dapat menentukan apakah operasi tersebut adalah penjumlahan bilangan bulat (menghasilkan instruksi OPR ADD) atau penggabungan string (yang mungkin memerlukan instruksi berbeda).

Informasi tambahan yang melekat pada setiap node DAST yang relevan mencakup:

- **Tipe Data (*SemanticType*)**: setiap node ekspresi dianotasi dengan tipe data hasil evaluasinya (*Integer, Real, Boolean, Char, String*). Informasi ini digunakan generator untuk memilih instruksi OPR yang tepat.
- **Referensi Tabel Simbol (*tab_index*)**: node yang merujuk pada identifier (variabel, fungsi, prosedur) memiliki referensi langsung ke entri dalam tabel simbol. Referensi ini memberikan informasi level leksikal dan alamat memori dari identifier tersebut, yang diperlukan oleh instruksi LOD dan STO.
- **Level Leksikal (*lexical_level*)**: menunjukkan kedalaman scope di mana sebuah identifier dideklarasikan. Informasi ini digunakan pada instruksi LOD dan STO untuk mengakses variabel pada scope yang berbeda (*non-local access*).
- **Alamat Memori (*address*)**: offset posisi variabel di dalam *stack frame* pada saat eksekusi. Alamat ini digunakan sebagai operand dari instruksi LOD (muat dari memori) dan STO (simpan ke memori).

5. *Interpreter dan Virtual Machine*

Setelah *Intermediate Code Generator* berhasil meratakan DAST menjadi daftar instruksi TAC, tahap selanjutnya adalah eksekusi. Tanggung jawab ini diemban oleh Interpreter, yang berperan sebagai sebuah *Virtual Machine* (VM). VM yang dibangun tidak memiliki wujud fisik, tetapi memiliki logika memori dan siklus kerja yang secara konseptual identik dengan CPU sungguhan.

Interpreter beroperasi dalam sebuah siklus eksekusi yang terdiri dari tiga tahap utama secara berulang:

- **Fetch**: membaca baris instruksi TAC yang saat ini ditunjuk oleh *Instruction Pointer* (IP). IP adalah sebuah penunjuk khusus, analog dengan *Program Counter* pada CPU sungguhan, yang mencatat baris instruksi yang sedang diproses.
- **Decode**: mengurai baris instruksi tersebut untuk mengidentifikasi jenis perintahnya (misalnya, apakah ini instruksi LIT, LOD, STO, OPR, atau JMP) operand-operandnya.

- **Execute:** melaksanakan aksi nyata sesuai instruksi yang telah didekode. Aksi ini dapat berupa mendorong nilai ke stack, mengambil nilai stack, melakukan operasi aritmatika atau logika, atau memindahkan IP ke posisi yang berbeda (*jump*).

Setelah tahap *Execute* selesai, IP secara otomatis maju ke baris berikutnya ($IP := IP + 1$), kecuali jika instruksi yang baru saja dieksekusi adalah JMP atau JPC yang memindahkan IP ke baris lain. Siklus ini terus berulang hingga interpreter mencapai instruksi RET pada level program utama, yang menandakan akhir dari eksekusi.

6. *Stack Machine dan Memori Eksekusi*

Arsitektur memori yang digunakan oleh Interpreter Arion adalah Stack Machine. Berbeda dengan arsitektur berbasis register, Stack Machine menggunakan sebuah tumpukan (*stack*) sebagai satu-satunya sumber penyimpanan nilai sementara maupun variabel program. Stack beroperasi berdasarkan prinsip LIFO (*Last In, First Out*), nilai yang terakhir dimasukkan ke dalam stack adalah nilai yang pertama dikeluarkan.

Dalam implementasi Arion, stack bersifat *flat* yang berupa array satu dimensi dan diakses menggunakan dua penunjuk utama:

- **Top (t):** penunjuk ke elemen paling atas stack. Setiap operasi PUSH menaikkan *t*, sedangkan setiap operasi POP menurunkan *t*.
- **Base (b):** penunjuk ke dasar atau awal dari *stack frame* yang sedang aktif. Base digunakan untuk mengakses variabel lokal dan parameter dalam *scope* yang sedang dieksekusi.

Setiap kali sebuah prosedur atau fungsi dipanggil, sebuah *stack frame* atau *activation record* baru dibuat di puncak stack. Setiap *stack frame* memiliki struktur yang tetap, terdiri dari:

- **Static Link (SL):** menunjuk ke *base* dari *stack frame* pada scope leksikal yang mengelilingi prosedur ini. Digunakan untuk mengakses variabel *non-local*
- **Dynamic Link (DL):** menunjuk ke *base* dari *stack frame* (caller). Digunakan untuk memulihkan nilai *base* saat prosedur selesai.
- **Return Address (RA):** menyimpan nomor baris instruksi TAC tempat eksekusi harus dilanjutkan setelah prosedur ini selesai.
- **Variabel Lokal:** ruang penyimpanan untuk semua variabel yang dideklarasikan di dalam prosedur atau program saat ini. Variabel pertama berada di offset +3 dari base, variabel kedua di offset +4, dan seterusnya.

Instruksi `INT 0 m` bertugas menginisiasi sebuah *stack frame* baru dengan mengalokasikan ruang sebesar m slot pada stack. Nilai m dihitung sebagai 3 (untuk SL, DL, RA) ditambah jumlah variabel yang dideklarasikan dalam blok tersebut.

7. Daftar Instruksi TAC Arion

Pada Arion Interpreter, *Intermediate Code* menggunakan delapan jenis instruksi dasar. Setiap instruksi memiliki format: `[nomor_baris] [instruksi] [level] [operand]`. Berikut adalah penjelasan dari masing-masing instruksi:

- **LIT l v:** (*Load Literal*) Mendorong nilai literal v ke puncak stack. Instruksi ini digunakan setiap kali terdapat nilai konstan (angka atau nilai boolean) dalam kode sumber.
- **LOD l a:** (*Load Value*) Memuat nilai dari alamat memori a pada level leksikal l mendorongnya ke puncak stack. Instruksi ini digunakan setiap kali sebuah variabel dibaca nilainya.
- **STO l a:** (*Store Value*) Mengambil nilai dari puncak stack dan menyimpannya ke alamat memori a pada level leksikal l . Instruksi ini digunakan untuk operasi penugasan (assignment).
- **CAL l f:** (*Call*) Memanggil prosedur atau fungsi yang dimulai pada baris f . Sebelum melompat, instruksi ini menyiapkan *stack frame* baru dengan mengisi SL dan DL.
- **INT l m:** (*Initiate Memory*) Menginisiasi *stack frame* baru dengan mengalokasikan ruang sebesar m slot. Instruksi ini selalu menjadi instruksi pertama dalam setiap blok program, prosedur, atau fungsi.
- **JMP l f:** (*Unconditional Jump*) Memindahkan *Instruction Pointer* (IP) ke baris f tanpa syarat apapun. Digunakan untuk melompati blok *else* atau mengakhiri iterasi *while*.
- **JPC l f:** (*Conditional Jump*) Mengambil nilai dari puncak stack dan memeriksa nilainya. Jika nilainya 0 (*false*), IP berpindah ke baris f . Jika nilainya 1 (*true*), eksekusi berlanjut ke instruksi berikutnya (*fall-through*). Digunakan untuk percabangan *if* dan kondisi *while*.
- **OPR l o:** (*Operation*) Menjalankan operasi bernomor o terhadap nilai-nilai di puncak stack. Ini adalah instruksi paling serbaguna, mencakup operasi aritmatika, komparasi, dan perintah output.
- **RET:** (*Return*) Keluar dari prosedur atau fungsi yang sedang berjalan. Instruksi ini memulihkan nilai *base* dan *IP* dari *stack frame* sebelumnya (*caller*), lalu melepas *stack frame* yang baru saja selesai dari stack.

8. Instruksi OPR dan Operasi Stack

Instruksi OPR adalah instruksi yang paling kaya fungsi dalam set instruksi Arion. Dengan satu jenis instruksi ini, interpreter dapat menjalankan 14 kategori operasi berbeda yang ditentukan oleh parameter nomor operasi. Berikut adalah pengelompokan operasi OPR berdasarkan kategorinya:

a. Operasi Aritmatika (Unary dan Binary)

- **NEG (1):** Negasi. Mengambil satu nilai dari puncak stack dan mendorong kembali nilai negasinya.
- **ADD (2):** Penjumlahan. Mengambil dua nilai dari puncak stack, menjumlahkannya, lalu mendorong hasilnya.
- **SUB (3):** Pengurangan. Mengambil dua nilai dari puncak stack, mengurangkan nilai teratas dari nilai di bawahnya, lalu mendorong hasilnya.
- **MUL (4):** Perkalian. Mengambil dua nilai dari puncak stack, mengalikannya, lalu mendorong hasilnya.
- **DIV (5):** Pembagian. Mengambil dua nilai dari puncak stack, membagi nilai di bawah puncak dengan nilai puncak, lalu mendorong hasilnya.
- **MOD (6):** Modulus. Mengambil dua nilai dari puncak stack, menghitung sisa bagi, lalu mendorong hasilnya.

b. Operasi Komparasi (Menghasilkan nilai Boolean: 1 atau 0)

- **EQL (7):** Mengecek apakah kedua nilai di puncak stack sama ($==$). Mendorong 1 jika sama, 0 jika tidak.
- **NEQ (8):** Mengecek apakah kedua nilai tidak sama ($!=$).
- **LSS (9):** Mengecek apakah nilai di bawah puncak lebih kecil dari nilai puncak ($<$).
- **GEQ (10):** Mengecek apakah nilai di bawah puncak lebih besar atau sama dengan nilai puncak ($>=$).
- **GTR (11):** Mengecek apakah nilai di bawah puncak lebih besar dari nilai puncak ($>$).
- **LEQ (12):** Mengecek apakah nilai di bawah puncak lebih kecil atau sama dengan nilai puncak ($<=$).

c. Operasi Output

- **WRT (13):** Mengambil nilai dari puncak stack dan menulisnya ke output standar tanpa menambahkan karakter newline.
- **WRTLN (14):** Mengambil nilai dari puncak stack dan menulisnya ke output standar kemudian menambahkan karakter newline ($\backslash n$) di akhir.

Instruksi ini berkaitan dengan pernyataan `writeln` pada kode sumber Arion.

9. *Pengelolaan Scope Leksikal dan Static Link*

Bahasa Arion, mendukung *nested procedure*, yaitu sebuah prosedur dapat dideklarasikan di dalam prosedur lain. Hal ini menciptakan hierarki scope leksikal yang bisa memiliki kedalaman lebih dari satu level. Sebuah prosedur dalam yang sedang berjalan harus mampu mengakses variabel dari prosedur luarnya, sebuah kemampuan yang tidak dapat dipenuhi hanya dengan menggunakan *dynamic link* saja.

Static Link adalah solusi untuk masalah ini. Saat sebuah prosedur dipanggil, *static link* dari *stack frame* baru tersebut diisi dengan alamat *base* dari *stack frame* yang secara leksikal mengelilinginya (bukan *stack frame* pemanggil). Dengan menelusuri rantai *static link* sebanyak ($\text{level_saat_ini} - \text{level_target}$) kali, interpreter dapat menemukan *base* dari *stack frame* yang menyimpan variabel yang ingin diakses.

Instruksi `LOD 1 a` dan `STO 1 a` menggunakan parameter *l* (*level difference*) untuk menentukan seberapa jauh interpreter harus menelusuri rantai *static link*. Jika $l = 0$, variabel berada pada *stack frame* yang sedang aktif (variabel lokal). Jika $l = 1$, interpreter harus menelusuri satu *static link* ke atas untuk menemukan *stack frame* yang tepat, dan seterusnya.

10. *Runtime Type Checking*

Runtime Type Checking adalah proses pemeriksaan kesesuaian tipe data yang dilakukan pada saat program sedang berjalan (*runtime*). Terdapat pertanyaan yang wajar muncul, mengapa pemeriksaan tipe masih diperlukan saat runtime, padahal analisis semantik sudah melakukan pemeriksaan tipe secara statis (*static type checking*)?

Static type checking memeriksa kepatuhan terhadap aturan tipe berdasarkan teks kode sumber sebelum program dijalankan, sedangkan *runtime type checking* berhadapan dengan nilai aktual yang ada di dalam memori saat program berjalan. Dalam bahasa yang bersifat *weakly-typed*, nilai sebuah variabel dapat berubah tipe saat runtime, sehingga pemeriksaan statis saja tidak cukup.

Namun, perlu dicatat bahwa bahasa Arion bersifat *strongly-typed*. Artinya, tipe setiap variabel sudah ditentukan secara pasti pada saat kompilasi dan tidak dapat berubah saat runtime. Dengan demikian, pada implementasi Milestone 4 ini, *runtime type checking* berstatus tidak diwajibkan. Semua pemeriksaan tipe yang diperlukan sudah dijamin tuntas oleh *semantic analyzer* pada fase sebelumnya. Meski demikian, pemahaman

konsep ini tetap penting sebagai landasan untuk memahami perbedaan antara *compile-time* dan *runtime* dalam konteks interpreter.

11. Kerentanan pada Interpreter

Interpreter yang baik tidak hanya mampu menjalankan kode yang benar, tetapi juga mampu bertahan ketika menghadapi kode yang salah, berbahaya, atau situasi eksekusi yang tidak terduga. Berbeda dengan bahasa tingkat rendah seperti C yang mendelegasikan tanggung jawab keamanan kepada programmer, sebuah interpreter yang berjalan di atas VM harus memiliki mekanisme perlindungan internal.

Beberapa kategori kerentanan yang perlu dipertimbangkan pada implementasi Interpreter Arion adalah:

a. Kerentanan pada Stack

- **Stack Overflow:** Terjadi ketika program terus-menerus mendorong (*push*) stack frame baru hingga melampaui batas memori yang dialokasikan. Kasus paling umum adalah *infinite recursion*, yaitu prosedur yang memanggil dirinya sendiri tanpa *base case*. Interpreter harus mampu mendeteksi kedalaman stack dan menghentikan eksekusi dengan pesan error jika melebihi batas, misalnya, maksimal 1000 frame.
- **Stack Underflow:** Terjadi ketika interpreter mencoba mengambil (*pop*) nilai dari stack yang sudah kosong. Kondisi ini biasanya mengindikasikan adanya cacat logika pada *Intermediate Code Generator*.
- **Stack Smashing:** Bentuk klasik dari *buffer overflow*. Jika sebuah variabel lokal, misalnya array, diizinkan untuk menulis data melebihi ukuran yang dialokasikan, data tersebut dapat meluber dan menimpa *return address* atau *dynamic link* pada stack frame yang sama, menyebabkan interpreter melompat ke lokasi memori yang acak saat fungsi selesai.
- **Stack Corruption:** Kondisi umum ketika struktur stack kehilangan keseimbangannya akibat jumlah operasi *push* dan *pop* yang tidak simetris, menyebabkan seluruh pembacaan variabel selanjutnya menjadi salah.

b. Kerentanan Akses Memori

- **Out-of-Bounds Variable Access:** Karena bahasa Arion mendukung tipe data array, interpreter wajib memvalidasi setiap akses indeks array. Jika sebuah array memiliki panjang n dengan indeks valid 0 sampai $n-1$, dan program mencoba mengakses indeks yang berada di luar rentang tersebut, interpreter harus

mencegat instruksi tersebut dan melempar *IndexOutOfBoundsException* daripada meneruskannya ke memori fisik komputer.

c. Kerentanan Alur Eksekusi

- **Invalid Jump Targets:** Jika *Intermediate Code Generator* menghasilkan instruksi JMP atau JPC yang menunjuk ke nomor baris yang tidak ada dalam daftar instruksi, interpreter harus mampu mendeteksi dan menangani kondisi ini dengan melaporkan error, bukan membiarkan IP tersesat ke area memori yang acak.

d. Kerentanan Aritmatika

- **Numerical Overflow/Underflow:** Setiap tipe data memiliki kapasitas nilai maksimum dan minimum. Misalnya, tipe *Integer* 32-bit memiliki nilai maksimum sekitar 2,14 miliar. Jika sebuah operasi menghasilkan nilai yang melampaui batas ini, terjadi fenomena *integer wrap-around* yang dapat menyebabkan logika program menjadi kacau. Interpreter yang tangguh harus mampu mendeteksi kondisi ini dan menghentikan dengan *OverflowError* atau *UnderflowError* yang informatif.

12. Hubungan Antar Komponen dalam Pipeline Arion

Milestone 4 merupakan tahap puncak dari keseluruhan *pipeline* Arion Compiler. Seluruh komponen yang dibangun pada milestone-milestone sebelumnya kini disatukan untuk pertama kalinya menjadi sebuah sistem yang utuh dan fungsional. Berikut adalah gambaran hubungan antar komponen:

- **Milestone 1 (Lexical Analyzer / Lexer):** Memecah teks kode sumber menjadi token-token bermakna. Menghasilkan *token stream* sebagai masukan bagi parser.
- **Milestone 2 (Syntax Analyzer / Parser):** Menyusun token-token menjadi *Parse Tree* atau *Abstract Syntax Tree* (AST) yang merepresentasikan struktur hierarkis program sesuai dengan *grammar* bahasa Arion.
- **Milestone 3 (Semantic Analyzer):** Menelusuri AST, mengisi tabel simbol, memeriksa kebenaran semantik (tipe data, scope), dan menghias setiap node AST dengan informasi semantik untuk menghasilkan *Decorated AST* (DAST).
- **Milestone 4 (Intermediate Code Generator):** Menelusuri DAST secara *depth-first* untuk menghasilkan daftar instruksi TAC yang bersifat linier.
- **Milestone 4 (Interpreter / Stack Machine):** Membaca daftar instruksi TAC dan mengeksekusinya satu per satu menggunakan Stack Machine, mengelola memori

melalui stack dengan mekanisme stack frame, dan menghasilkan output akhir program.

BAB 2

PERANCANGAN DAN IMPLEMENTASI

1. Perancangan

1.1. Arsitektur Umum

Pada Milestone 4, Arion Compiler dikembangkan dengan dua komponen baru pada fase back-end: Intermediate Code Generator dan Interpreter. Dua komponen ini menjadikan pipeline yang sudah ada menjadi lebih lengkap sehingga program Arion tidak hanya divalidasi secara sintaksis dan semantis, tetapi juga benar-benar dapat dijalankan dan menghasilkan output nyata.

Alur kerja pipeline secara keseluruhan menjadi:

- **Lexer**: membaca source code dan menghasilkan token.
- **Parser**: menyusun token menjadi Parse Tree.
- **ASTBuilder**: mengonversi Parse Tree menjadi Abstract Syntax Tree (AST) yang lebih ringkas.
- **SemanticAnalyzer**: menelusuri AST, mengisi tabel simbol, dan menghasilkan Decorated AST (DAST) lengkap dengan anotasi tipe dan referensi simbol.
- **CodeGenerator**: membaca DAST dan menghasilkan daftar instruksi Intermediate Code berbasis stack yang disimpan di dalam CodeBuffer.
- **Interpreter**: membaca CodeBuffer instruksi demi instruksi, mensimulasikan eksekusi pada virtual machine berbasis stack, dan menghasilkan output program.

Pemisahan antara fase front-end (Lexer hingga SemanticAnalyzer) dan back-end (CodeGenerator dan Interpreter) membuat sistem ini modular. Selama DAST yang dihasilkan konsisten, back-end tidak perlu tahu detail sintaksis bahasa Arion sama sekali.

Selain menerima source code Arion secara langsung, program juga mendukung mode input DAST: jika file input dideteksi oleh DecoratedASTReader sebagai output dari Milestone 3, fase front-end dilewati sepenuhnya dan eksekusi langsung dimulai dari CodeGenerator.

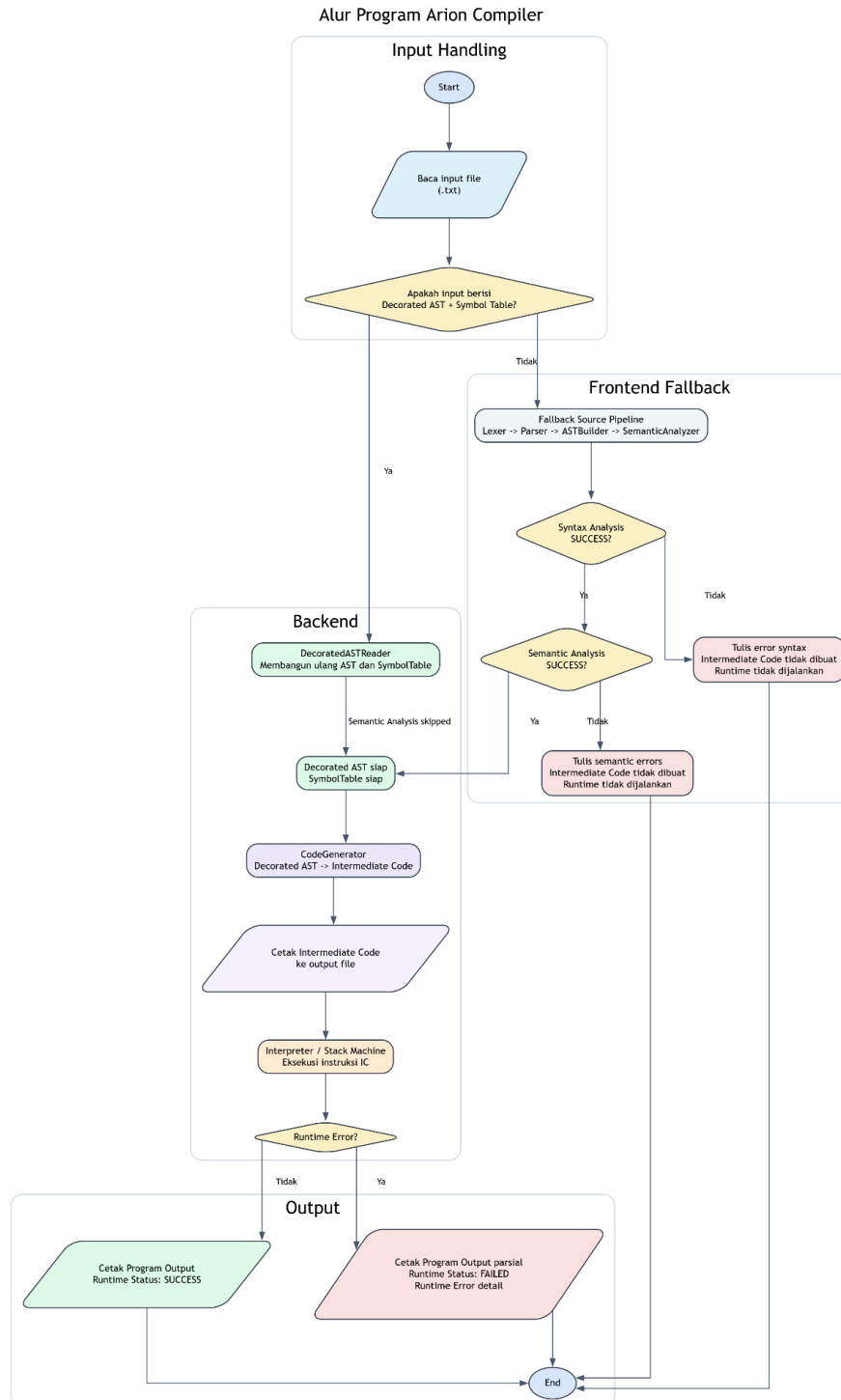
1.2. Struktur Program

Program dikembangkan menggunakan bahasa C++ dan menambahkan beberapa file baru ke dalam folder src/ yang sudah ada dari Milestone 3. Berikut adalah daftar file yang ditambahkan atau dimodifikasi beserta tugasnya:

Tabel 1 Struktur Program

File	Tanggung Jawab
IntermediateCode.hpp / .cpp	Mendefinisikan enum OpCode (INT, LIT, LOD, STO, OPR, JMP, JPC, CAL, RET, HALT), struct Instruction, dan kelas CodeBuffer. Menyediakan emit(), patch(), currentAddress(), dan print().
CodeGenerator.hpp / .cpp	Kelas CodeGenerator yang membaca DAST dan SymbolTable lalu menghasilkan instruksi ke CodeBuffer. Menyediakan generate() sebagai titik masuk serta fungsi generateXxx() per jenis node AST.
Interpreter.hpp / .cpp	Kelas Interpreter yang bertindak sebagai virtual machine. Membaca CodeBuffer dan mengeksekusi instruksi satu per satu menggunakan stack. Mengelola pc_, sp_, bp_, dan menangani runtime error.
RuntimeValue.hpp / .cpp	Mendefinisikan tipe Value sebagai <code>std::variant<int,double,char,bool,string></code> beserta fungsi-fungsi operasi aritmatika, relasional, dan logika yang bekerja di atas tipe tersebut.
RuntimeErrors.hpp / .cpp	Mendefinisikan enum RuntimeErrorType, struct RuntimeError, dan kelas RuntimeErrorManager yang mengumpulkan error runtime selama eksekusi berlangsung.
DecoratedASTReader.hpp / .cpp	Membaca file output Milestone 3 (berisi DAST + tabel simbol) sebagai input alternatif, sehingga back-end bisa diuji langsung tanpa menjalankan ulang front-end.
Main.cpp	Entry point program yang diperbarui. Mengintegrasikan seluruh komponen baru.

1.3 Alur Program



Flowchart menggambarkan arsitektur kompiler Arion secara menyeluruh, dimulai dari pembacaan file input berisi kode sumber atau representasi Decorated AST beserta

Symbol Table. Jika input yang diberikan sudah dalam bentuk decorated AST dan symbol table, sistem akan langsung menggunakan modul DecoratedASTReader untuk membangun kembali struktur tersebut tanpa perlu melalui pipeline frontend. Sebaliknya, jika input merupakan file teks kode sumber Arion, maka compiler akan menjalankan Fallback Source Pipeline yang terdiri dari empat tahap utama, yaitu Lexer (analisis leksikal untuk memecah source code menjadi token), Parser (analisis sintaksis untuk membangun parse tree), ASTBuilder (transformasi parse tree menjadi Abstract Syntax Tree), dan SemanticAnalyzer (pemeriksaan semantik, type checking, serta pembangunan symbol table). Kedua jalur ini pada akhirnya menghasilkan Decorated AST yang valid dan Symbol Table yang lengkap, yang menjadi prasyarat untuk masuk ke bagian backend compiler.

Tahapan Backend dimulai dengan CodeGenerator yang menerjemahkan Decorated AST menjadi Intermediate Code (IC) instruksi yang kemudian dicetak ke file output. Selanjutnya, Interpreter / Stack Machine mengeksekusi instruksi IC tersebut secara berurutan. Setelah eksekusi selesai, sistem melakukan pengecekan apakah terjadi Runtime Error. Jika tidak ada error, program akan mencetak output lengkap dengan status SUCCESS dan menyelesaikan proses; namun jika terjadi error saat runtime, sistem akan mencetak output secara parsial yang sudah dihasilkan sebelum error, menampilkan status FAILED beserta detail error-nya, lalu mengakhiri eksekusi. Sepanjang pipeline, compiler ini memiliki mekanisme penanganan error yang jelas sehingga setiap kegagalan pada tahapan tertentu akan langsung menghentikan eksekusi lebih lanjut dan memberikan informasi error yang relevan kepada pengguna.

2. Implementasi

2.1. Kakas yang Digunakan

Implementasi program menggunakan kakas yang sama seperti pada Milestone 2:

- Bahasa Pemrograman: C++, alasan pemilihan adalah karena kemampuan manajemen memori manual yang baik untuk membangun struktur pohon (*tree*), serta adanya *standar library* yang memadai seperti `std::vector`, `std::string`, dan lain sebagainya. Selain itu, di C++, kita dapat menggunakan Object Oriented Programming untuk handle struktur data y
- Compiler: g++ (GNU Compiler Collection) yang mendukung standar C++17.

2.2. Kelas CodeBuffer dan Struktur Instruksi

CodeBuffer adalah wadah utama untuk instruksi Intermediate Code. Setiap instruksi direpresentasikan oleh struct Instruction dengan empat field:

```

enum class OpCode { INT, LIT, LOD, STO, OPR, JMP,
JPC, CAL, RET, HALT };

struct Instruction {
    OpCode   op;           // jenis operasi
    int      level = 0;    // selisih level
                        leksikal (LOD/STO/CAL)
    int      operand = 0;  // nilai literal, alamat
                        memori, atau nomor OPR
    std::string comment;   // keterangan untuk
                        output debugging
};

```

CodeBuffer menyediakan empat fungsi utama:

- emit(op, level, operand, comment): menambah satu instruksi ke daftar dan mengembalikan indeksinya (alamat instruksi). Nilai ini penting untuk patching nanti.
- patch(index, operand): memperbarui field operand instruksi di alamat index. Digunakan untuk mengisi target JMP/JPC yang belum diketahui saat pertama kali di-emit.
- currentAddress(): mengembalikan jumlah instruksi saat ini, berguna sebagai acuan alamat sebelum emit.
- print(out): mencetak semua instruksi ke output stream dengan format: 'alamat OPCODE level operand ; komentar'.

2.3. Kelas CodeGenerator

CodeGenerator bertanggung jawab mengubah DAST menjadi instruksi Intermediate Code di dalam CodeBuffer. Ia menerima SymbolTable dari SemanticAnalyzer untuk membaca alamat memori, level leksikal, dan ukuran frame setiap entri.

generate() dan generateNode() - Dispatcher Utama

Titik masuk adalah generate(), yang menginisialisasi CodeBuffer kosong lalu memanggil generateNode() pada node root. generateNode() adalah dispatcher yang meneruskan ke fungsi generateXxx() berdasarkan ASTKind:

```
void CodeGenerator::generateNode (ASTNode* node) {
    switch (node->kind) {
        case ASTKind::Program:
generateProgram(node);          break;
        case ASTKind::Assign:
generateAssign(node);           break;
        case ASTKind::If:
generateIf(node);               break;
        case ASTKind::While:
generateWhile(node);            break;
        case ASTKind::Repeat:
generateRepeat(node);           break;
        case ASTKind::For:
generateFor(node);              break;
        case ASTKind::Case:
generateCase(node);             break;
        case ASTKind::ProcedureCall:
generateProcedureCall(node);    break;
        case ASTKind::FunctionCall:
generateFunctionCall(node);     break;
        case ASTKind::ProcDecl:
        case ASTKind::FuncDecl:
generateSubprogramDecl(node); break;
        // semua jenis ekspresi →
generateExpression:
        case ASTKind::BinOp: case ASTKind::UnaryOp:
        case ASTKind::Literal: case ASTKind::Var:
        case ASTKind::ArrayAccess: case
ASTKind::FieldAccess:
            generateExpression(node); break;
        default: break;
    }
}
```

generateProgram() - Inisialisasi Frame Program

Fungsi ini menghitung ukuran frame program utama dengan membaca semua variabel dan parameter level 0 dari SymbolTable. Ukuran frame = FRAME_HEADER (3 slot untuk static link, dynamic link, return address) + offset terbesar dari semua variabel global. Hasilnya di-emit sebagai INT, kemudian semua child di-generate, dan diakhiri dengan RET:

```
void CodeGenerator::generateProgram(ASTNode* node)
{
    int frameSize = FRAME_HEADER; // FRAME_HEADER
    = 3
    int maxEndAdr = 0;
    for (int i = 0; i < sym_.tabSize(); ++i) {
        const TabEntry& e = sym_.tabAt(i);
        if (e.lev == 0 && (e.obj == Variable ||
e.obj == Parameter)) {
            int endAdr = e.adr + typeSize(e.type);
            if (endAdr > maxEndAdr) maxEndAdr =
endAdr;
        }
    }
    frameSize += maxEndAdr;
    buf_.emit(OpCode::INT, 0, frameSize, "alokasi
frame program");
    for (ASTNode* child : node->children)
generateNode(child);
    buf_.emit(OpCode::RET, 0, 0, "akhir program");
}
```

generateSubprogramDecl() - Prosedur dan Fungsi

Deklarasi prosedur/fungsi di-emit dengan teknik JMP-skip: emit JMP dengan operand sementara untuk melewati seluruh body, emit body subprogram, lalu patch JMP ke instruksi setelah RET. Dengan cara ini body subprogram hanya dieksekusi saat dipanggil lewat CAL, bukan saat program pertama kali berjalan.

Untuk menangani forward call (subprogram dipanggil sebelum dideklarasikan dalam kode), CodeGenerator menyimpan dua struktur: subprogramAddr_ (tabIndex → alamat body) dan pendingSubprogramCalls_ (tabIndex → daftar alamat instruksi CAL yang perlu di-patch). Begitu body selesai di-emit, seluruh CAL yang pending langsung di-patch secara otomatis:

```

void CodeGenerator::generateSubprogramDecl(ASTNode*
node) {
    int jmpSkip = buf_.emit(OpCode::JMP, 0, 0, "jmp
skip body " + node→value);

    int bodyStart = buf_.currentAddress();
    subprogramAddr_[ti] = bodyStart;
    // patch semua pending CAL yang sudah di-emit
    sebelumnya (forward call)
    for (int callAddr :
pendingSubprogramCalls_[ti])
        buf_.patch(callAddr, bodyStart);

    buf_.emit(OpCode::INT, currentLevel_+1,
frameSize, "alokasi frame " + node→value);
    currentLevel_++;
    for (ASTNode* child : node→children) //
generate body
        if (child→kind == ASTKind::Block)
generateBlock(child);
        currentLevel_--;
    buf_.emit(OpCode::RET, 0, 0, "ret dari " +
node→value);
    buf_.patch(jmpSkip, buf_.currentAddress()); //
patch JMP skip
}

```

generateAssign() dan LValue Resolution

Assignment memerlukan pengetahuan alamat memori sisi kiri (LValue). Fungsi `resolveLValue()` mengembalikan struct `LValue{level, address, isStack}`. Untuk variabel scalar, level dan address dibaca langsung dari `SymbolTable` dengan `address = e.adr + FRAME_HEADER`. Untuk array, alamat dihitung secara dinamis: `resolveLValueArrayAccess()` meng-emit serangkaian instruksi aritmatika

yang meninggalkan alamat elemen di stack (isStack = true), dan STO digunakan dengan operand -1 untuk indirect store. Contoh nyata untuk `arr[i] := val`:

```
// Hasil emit untuk: arr[i] := val (arr di offset
5, low=1, elsz=1)
; -- generate RHS (val) --
LOD 0 3    ; load val
; -- resolveLValueArrayAccess: hitung alamat arr[i]
--
LIT 0 5     ; array base addr (LValue)
LOD 0 3     ; load i (ekspresi indeks)
LIT 0 1     ; array low (LValue)
OPR 0 3     ; idx - low
OPR 0 2     ; base + offset = abs addr
STO 0 -1    ; indirect store: pop addr, pop val,
stack[addr] = val
```

generateExpression() - Berbagai Jenis Ekspresi

Semua ekspresi bekerja dengan prinsip: push hasilnya ke stack. `generateExpression()` mendispatch ke fungsi yang sesuai berdasarkan jenis node:

Tabel 2 Cara Generasi Kode per Jenis Ekspresi

Jenis Ekspresi	Instruksi yang Dihasilkan
Literal Integer / Boolean / Char	LIT 0 nilai Contoh: literal 5 → LIT 0 5, literal true → LIT 0 1
Literal String	LIT 0 -1 dengan komentar 'string:isistring' Operand -1 + prefix 'string:' adalah penanda khusus agar Interpreter mem-push nilai string ke stack.
Literal Real	LIT 0 nilai (di-truncate ke int, karena stack menggunakan slot integer)
Variabel (obj bukan Constant)	LOD level (e.adr + FRAME_HEADER) level = currentLevel_ - e.lev dari

	SymbolTable
Konstanta (obj == Constant)	LIT 0 e.adr Nilai konstanta disimpan di field adr pada SymbolTable, bukan di slot memori stack.
BinOp (a op b)	generate(a) → generate(b) → OPR 0 kode_operasi Kedua operand di-push ke stack, lalu OPR mengambil keduanya dan push hasilnya.
UnaryOp (-x atau not x)	generate(x) → OPR 0 1 (NEG) atau OPR 0 17 (NOT)
ArrayAccess arr[i] (rvalue)	LIT base_addr → generate(i) → LIT low → OPR SUB → OPR ADD → LOD level -1 Alamat dihitung dinamis di runtime. LOD -1 berarti indirect load: pop alamat dari stack, push nilai dari stack[addr].
FieldAccess rec.field (rvalue)	LOD level (base.adr + field.adr + FRAME_HEADER) Offset field sudah diketahui statis dari SymbolTable sehingga alamat bisa dihitung saat kompilasi.
FunctionCall f(args)	generate(arg1) → generate(arg2) → ... → CAL level bodyAddr → LOD level (f.adr + FRAME_HEADER) (baca return value)

Control Flow (IF, WHILE, REPEAT, FOR, CASE)

Semua struktur kontrol menggunakan teknik emit-then-patch: instruksi lompatan di-emit dengan operand sementara (0), isi blok di generate, lalu operand instruksi lompatan di patch ke alamat yang benar setelah blok selesai.

Tabel 3 Pola Instruksi Setiap Struktur Kontrol

Konstruksi	Pola Instruksi yang Dihasilkan
IF-THEN	generate(kondisi) → JPC ke AFTER → generate(then) → AFTER:
IF-THEN-ELSE	generate(kondisi) → JPC ke ELSE → generate(then) → JMP ke END → ELSE: generate(else) → END:

WHILE	LOOP: generate(kondisi) → JPC ke END → generate(body) → JMP ke LOOP → END:
REPEAT-UNTIL	LOOP: generate(body) → generate(kondisi_until) → JPC ke LOOP (lompat kembali ke awal jika kondisi belum terpenuhi)
FOR TO	generate(init) → STO ctrl_var LOOP: LOD ctrl → generate(limit) → OPR LEQ → JPC ke END generate(body) → LOD ctrl → LIT 1 → OPR ADD → STO ctrl → JMP ke LOOP → END:
FOR DOWNT0	Sama dengan FOR TO, tetapi kondisi pakai OPR GEQ dan step pakai OPR SUB

generateProcedureCall() dan generateFunctionCall()

Built-in writeln, write, readln, read ditangani secara khusus tanpa instruksi CAL:

- **writeln/write:** tiap argumen di-generate ke stack, lalu diikuti OPR WRT (13) atau OPR WRTLN (14) pada argumen terakhir.
- **readln/read:** untuk tiap target, emit LIT berisi lexical level, emit alamat target (atau resolveLValue untuk array), lalu OPR READLN (19) / OPR READ (18). Interpreter akan pop level + addr dari stack dan menyimpan nilai input ke lokasi tersebut.

Untuk prosedur/fungsi pengguna, emitSubprogramCall() meng-generate argumen ke stack lalu emit CAL dengan level leksikal dan alamat body. Jika body belum di-emit (forward call), alamat 0 digunakan sementara dan masuk ke pendingSubprogramCalls_ untuk di-patch saat body selesai.

Perbedaan antara ProcedureCall dan FunctionCall: setelah CAL selesai, FunctionCall menambahkan instruksi LOD untuk mengambil return value dari slot nama fungsi di frame pemanggil:

```
void CodeGenerator::generateFunctionCall(ASTNode* node) {
    // emit argumen + CAL
    emitSubprogramCall(node, funcEntry);
    // baca return value dari slot f di frame pemanggil
```

```

int diff = currentLevel_ - funcEntry.lev;
buf_.emit(OpCode::LOD, diff, funcEntry.adr + FRAME_HEADER,
          "load return value " + node->value);
}

```

Tabel 4 Pemetaan Operator ke Kode OPR

Operator Arion	Kode OPR	Keterangan
-x (unary)	1 (NEG)	Negasi satu nilai teratas stack
+	2 (ADD)	Penjumlahan
-	3 (SUB)	Pengurangan
*	4 (MUL)	Perkalian
/ atau div	5 (DIV)	Pembagian (deteksi DivisionByZero)
mod	6 (MOD)	Modulus (deteksi DivisionByZero)
=	7 (EQL)	Sama dengan
<>	8 (NEQ)	Tidak sama dengan
<	9 (LSS)	Kurang dari
>=	10 (GEQ)	Lebih dari atau sama dengan
>	11 (GTR)	Lebih dari
<=	12 (LEQ)	Kurang dari atau sama dengan
write (satu arg)	13 (WRT)	Cetak ke output tanpa newline
writeln (arg terakhir)	14 (WRTLN)	Cetak ke output dengan newline
and	15 (AND)	Logika AND
or	16 (OR)	Logika OR
not	17 (NOT)	Logika NOT
read	18 (READ)	Baca satu nilai dari stdin

readln	19 (READLN)	Baca satu nilai dari stdin
--------	-------------	----------------------------

2.4. Kelas RuntimeValue

Tipe Value merepresentasikan satu slot di stack runtime. Ia didefinisikan menggunakan std::variant sehingga satu slot bisa menyimpan nilai dari berbagai tipe data:

```
using Value = std::variant<int, double, char, bool,
std::string>;
```

Fungsi konversi (toInt, toReal, toBool, toString) mendukung widening otomatis: int ke double, char ke int, bool ke int (1/0). Fungsi operasi aritmatika mempromosikan ke double jika salah satu operand bertipe double, dan ke string jika operasi + melibatkan string. Fungsi divValues dan modValues melempar std::runtime_error dengan pesan 'Division by zero' yang akan ditangkap oleh Interpreter dan diubah menjadi RuntimeError:

```
Value divValues(const Value& a, const Value& b) {
    if (isReal(a) || isReal(b)) {
        double br = toReal(b);
        if (br == 0.0) throw
std::runtime_error("Division by zero");
        return toReal(a) / br;
    }
    int bi = toInt(b);
    if (bi == 0) throw std::runtime_error("Division
by zero");
    return toInt(a) / bi;
}
```

Fungsi isTruthy() digunakan oleh instruksi JPC: nilai dianggap false (dan JPC akan melompat) jika nilai tersebut adalah bool false, integer 0, double 0.0, atau string kosong.

2.5. Kelas Interpreter

Interpreter bertindak sebagai virtual machine (VM) yang mengeksekusi instruksi dari CodeBuffer menggunakan model stack machine.

State Virtual Machine

- **pc_ (Program Counter):** indeks instruksi yang sedang dieksekusi. Dinaikkan otomatis setiap siklus kecuali saat JMP/JPC yang langsung mengubah nilainya.
- **sp_ (Stack Pointer):** menunjuk ke slot paling atas stack yang terisi. Nilai -1 berarti stack kosong. push() menaikkan sp_ lalu mengisi, pop() membaca lalu menurunkan sp_.
- **bp_ (Base Pointer):** menunjuk ke awal stack frame aktif saat ini. Diperbarui setiap CAL dan dikembalikan ke nilai lama setiap RET.
- **stack_ (std::vector<Value>):** array dinamis yang menjadi memori utama VM, dengan batas maksimum MAX_STACK = 10.000 slot.

Siklus Eksekusi (Fetch-Decode-Execute)

Fungsi run() menjalankan loop utama VM. Setiap iterasi: fetch instruksi yang ditunjuk pc_, naikan pc_, lalu execute. Loop berhenti ketika pc_ melampaui panjang daftar instruksi, terjadi runtime error, atau jumlah langkah melampaui MAX_STEPS = 1.000.000 untuk mencegah infinite loop:

```
bool Interpreter::run() {
    pc_ = 0; sp_ = -1; bp_ = 0; steps_ = 0;
    while (pc_ < instrs.size()) {
        if (steps_++ ≥ MAX_STEPS) {
            errorMgr_.addError(ExecutionStepLimit,
"Execution step limit exceeded", pc_);
            return false;
        }
        const Instruction& instr = instrs[pc_++]; //
fetch + advance pc
        executeInstruction(instr);
        if (errorMgr_.hasErrors()) return false;
    }
    return true;
}
```

Eksekusi Setiap Instruksi (executeInstruction)

Tabel 5. Cara Eksekusi Setiap OpCode di Interpreter

OpCode	Yang Dilakukan Interpreter
INT 0 n	Alokasikan n slot baru di stack (sp_ += n) dan

	inisialisasi semuanya ke 0. Dipakai saat program atau subprogram mulai berjalan.
LIT 0 n	Push nilai integer n ke stack. Kasus khusus: jika n = -1 dan komentar diawali 'string:', push string yang ada setelah prefix tersebut sebagai Value bertipe string.
LOD level addr	Hitung alamat = base(level) + addr. Push stack[addr] ke stack. Kasus khusus: jika addr = -1, pop satu nilai dari stack dan gunakan sebagai alamat (indirect load untuk array).
STO level addr	Pop satu nilai dari stack. Hitung alamat = base(level) + addr. Simpan ke stack[addr]. Kasus khusus: jika addr = -1, pop juga alamat dari stack terlebih dahulu (indirect store untuk array).
OPR 0 n	Jalankan operasi ke-n (lihat Tabel 4). Unary: modifikasi stack[sp_] langsung. Biner: pop dua nilai, hitung, push hasilnya. WRT/WRTLN: pop dan cetak ke output_. READ/READLN: pop addr dan level, baca stdin, simpan ke stack[target].
JMP 0 target	Set pc_ = target (lompat tanpa syarat). Validasi target terlebih dahulu; jika di luar range instruksi, emit InvalidJump error.
JPC 0 target	Pop satu nilai. Jika !isTruthy(nilai) (false/0), set pc_ = target. Jika truthy, lanjut ke instruksi berikutnya (fall-through).
CAL level addr	Ambil argumen dari stack (sebanyak psze dari btab via addrToTabIndex). Push: static link = base(level), dynamic link = bp_, return address = pc_. Copy argumen ke atas header. Update bp_ ke frame baru. Set pc_ = addr.
RET 0 0	Baca return address dari stack[bp_+2] dan old bp_ dari stack[bp_+1]. Kembalikan sp_ = bp_-1, bp_ = oldBp, pc_ = returnAddr. Jika bp_ == 0, program utama selesai.

Fungsi base() untuk Lexical Scoping

Fungsi `base(level)` menghitung alamat base dari frame yang tepat berdasarkan selisih level leksikal. Ia menelusuri static link sebanyak level kali mulai dari `bp_` saat ini:

```
int Interpreter::base(int level) const {
    int b = bp_;
    while (level > 0) {
        b = toInt(stack_[b]); // stack[bp_] = static
        link ke frame scope luar
        level--;
    }
    return b;
}
```

Jika `level = 0`, fungsi mengembalikan `bp_` langsung (akses ke variabel lokal frame saat ini). Jika `level = 1`, ia mengikuti satu static link ke frame scope satu tingkat di luar. Mekanisme ini memungkinkan fungsi bersarang (nested function) mengakses variabel dari scope induknya tanpa menyentuh scope lain yang tidak relevan.

Struktur Stack Frame

Setiap kali instruksi CAL dieksekusi, tiga slot header di-push ke stack sebelum slot variabel lokal. Nilai `FRAME_HEADER = 3` mencerminkan tiga slot ini:

- **Slot `bp_+0`:** Static Link: hasil `base(level)`, yaitu alamat base frame scope leksikal satu tingkat di luar subprogram ini. Digunakan oleh `base()` untuk resolusi variabel dari scope luar.
- **Slot `bp_+1`:** Dynamic Link: nilai `bp_` sebelum pemanggilan. Dikembalikan oleh RET agar `bp_` kembali ke frame pemanggil.
- **Slot `bp_+2`:** Return Address: nilai `pc_` tepat setelah CAL (instruksi berikutnya di pemanggil). Dikembalikan oleh RET agar eksekusi melanjutkan dari sana.

Slot-slot selanjutnya diisi oleh INT di awal body subprogram untuk parameter dan variabel lokal. Semua offset variabel dari SymbolTable selalu ditambah `FRAME_HEADER` agar tidak bertabrakan dengan slot header. Berikut ilustrasi stack saat fungsi `DoubleValue(x)` dipanggil dengan argumen 7:

```
// Stack setelah CAL pada: result := DoubleValue(7)
```

```

// bp_ menunjuk ke awal frame DoubleValue
bp_+0 : static link (= base(0) = bp_ pemanggil
program utama)
bp_+1 : dynamic link (= bp_ pemanggil)
bp_+2 : return addr (= pc_ setelah CAL)
bp_+3 : x = 7          (parameter pertama, offset 0 dari
SymbolTable + FRAME_HEADER = 3)
bp_+4 : DoubleValue (slot return value fungsi,
offset 1 + 3 = 4)

```

Penanganan Argumen pada Instruksi CAL

Implementasi CAL pada Interpreter tidak sesederhana push header lalu lompat. Sebelum header di-push, argumen yang sudah ada di stack diambil terlebih dahulu (pop sebanyak psze, disimpan sementara). Setelah header di-push, argumen di-copy kembali ke atas header. Ini penting agar offset parameter di frame baru sesuai dengan offset yang ditetapkan SymbolTable. Jumlah psze dibaca menggunakan peta addrToTabIndex yang diteruskan dari Main.cpp:

```

case OpCode::CAL: {
    // 1. ambil argumen yang sudah di-push
    CodeGenerator
        std::vector<Value> args;
        for (int i = 0; i < psze; ++i)
            args.push_back(stack_[sp_ - psze + 1 + i]);
        sp_ -= psze;
    // 2. push header frame baru
        push(base(instr.level)); // static link
        push(bp_);               // dynamic link
        push(pc_);               // return address
    // 3. copy argumen ke atas header
        for (auto& arg : args) push(arg);
        bp_ = sp_ - psze - 2;    // bp_ ke slot static
link
        pc_ = instr.operand;    // lompat ke body
        break;
}

```

2.6. Kelas RuntimeExceptionManager

RuntimeExceptionManager mengumpulkan error yang terjadi saat runtime. Error ditambahkan via `addError(type, message, address)`. Begitu ada satu error masuk, `hasErrors()` mengembalikan true dan loop `run()` langsung menghentikan eksekusi.

Tabel 6. Tipe Runtime Error yang Ditangani

RuntimeExceptionType	Penyebab dan Kondisi Pemicu
StackOverflow	<code>sp_ + 1 >= MAX_STACK</code> saat push. Dipicu oleh rekursi tak terbatas (contoh: prosedur yang memanggil dirinya sendiri tanpa base case), terlihat di test case infinite recursion.
StackUnderflow	<code>pop()</code> atau <code>top()</code> dipanggil saat <code>sp_ < 0</code> . Menandakan CodeGenerator meng-emit pop yang tidak memiliki push pasangan.
InvalidJump	Target JMP atau JPC berada di luar range daftar instruksi (<code>< 0</code> atau <code>>=</code> jumlah instruksi). Menandakan kesalahan patching di CodeGenerator.
InvalidMemoryAccess	Alamat pada LOD atau STO berada di luar range vector <code>stack_</code> . Dipicu saat mengakses array di luar batas, terlihat di test case out-of-bounds access.
DivisionByZero	Operan pembagi pada OPR DIV atau OPR MOD bernilai 0. Dikembalikan dari <code>divValues()/modValues()</code> sebagai exception, lalu ditangkap dan diubah menjadi RuntimeException. Operan pembagi pada OPR DIV atau OPR MOD bernilai 0. Dikembalikan dari <code>divValues()/modValues()</code> sebagai exception, lalu ditangkap dan diubah menjadi RuntimeException.
InvalidCallFrame	<code>bp_ < 0</code> atau <code>bp_ + 2 >=</code> ukuran stack saat RET, menandakan frame call sudah korup.
ExecutionStepLimit	Langkah eksekusi melampaui <code>MAX_STEPS = 1.000.000</code> . Mekanisme proteksi untuk mendeteksi infinite loop, terlihat di test case infinite while loop.
InvalidInstruction	OpCode atau nomor OPR yang tidak dikenali.

2.7. Kelas DecoratedASTReader

DecoratedASTReader memungkinkan program menerima file output Milestone 3 (yang sudah berisi DAST dan tabel simbol) sebagai input langsung, tanpa perlu menjalankan ulang seluruh pipeline front-end.

Fungsi canRead() memeriksa apakah file mengandung penanda 'Decorated AST:' dan 'Symbol Table - tab:' secara berurutan. Jika ya, fungsi read() menjalankan parser berbasis state machine yang memproses file baris per baris melalui empat seksi: AST, Tab, BTab, dan ATab.

Pada bagian AST, kedalaman indentasi setiap baris (dihitung dari karakter tree-drawing seperti '├──' dan '└──') digunakan untuk menentukan posisi node dalam pohon. Metadata di dalam kurung siku ([type=..., tab=..., block=..., level=...]) di-parse untuk mengisi atribut dekorasi setiap ASTNode. Pada seksi Tab, setiap baris diurai menjadi entri TabEntry lengkap termasuk penanganan tipe Array/Record yang memiliki suffix [ref=N]. Hasil akhir dikembalikan sebagai DecoratedASTInput yang berisi ASTNode* root dan SymbolTable yang sudah terisi penuh:

```
bool DecoratedASTReader::canRead(const std::string&
filename) const {
    // cek apakah file mengandung kedua penanda wajib
    bool hasAST = false, hasTab = false;
    while (getline(in, line)) {
        if (trim(line) == "Decorated AST:")
hasAST = true;
        if (trim(line) == "Symbol Table - tab:")
hasTab = true;
        if (hasAST && hasTab) return true;
    }
    return false;
}
```

Ketika mode DAST aktif, output program akan memuat baris 'Input Mode: DECORATED AST' dan 'Semantic Analysis: SKIPPED (Decorated AST input)'. Mode ini digunakan oleh seluruh test case Milestone 4 yang menggunakan file output dari Milestone 3 sebagai input.

2.8. Alur Kerja Main.cpp

Main.cpp mengintegrasikan seluruh komponen dalam urutan berikut:

- Program mencoba membaca input sebagai DAST via DecoratedASTReader. Jika berhasil, langsung masuk ke generateAndRun() tanpa melewati front-end.
- Jika bukan DAST, jalankan pipeline lengkap: Lexer → Parser → ASTBuilder → SemanticAnalyzer.
- Jika semantic analysis gagal, tulis daftar error ke output dan hentikan tanpa generate kode.
- Jika berhasil, cetak DAST dan tabel simbol, lalu panggil generateAndRun().
- generateAndRun() menjalankan CodeGenerator, mencetak Intermediate Code, menjalankan Interpreter, lalu mencetak output program beserta status runtime dan daftar runtime error jika ada.

Cuplikan fungsi generateAndRun():

```
bool generateAndRun(ASTNode* astRoot, const
SymbolTable& symbols, ofstream& out) {
    CodeGenerator cg(symbols);
    CodeBuffer buf = cg.generate(astRoot);

    out << "Intermediate Code:\n";
    buf.print(out);

    // peta addr body → tabIndex (untuk Interpreter
membaca psze argumen)
    unordered_map<int,int> addrToTabIndex;
    for (const auto& kv : cg.subprogramAddr())
        addrToTabIndex[kv.second] = kv.first;

    Interpreter interp(buf, &symbols,
&addrToTabIndex);
    bool ok = interp.run();

    out << "Program Output:\n" << interp.output();
    out << "Runtime Status: " << (ok ? "SUCCESS" :
"FAILED") << "\n";
```

```

        if (!ok) {
            out << "Runtime Error:\n";
            for (const auto& err : interp.errors())
                out << "- " << err.message << " at
instruction "
                                << err.instructionAddress << "\n";
        }
        return ok;
    }
}

```

Peta `addrToTabIndex` diteruskan ke Interpreter agar saat instruksi CAL dieksekusi, Interpreter bisa mengetahui jumlah parameter (psze dari `btab`) yang perlu dipindahkan ke frame baru berdasarkan `SymbolTable`, bukan hanya dari informasi yang tersimpan di dalam `CodeBuffer`.

2.9. Implementasi Lanjutan (selain spesifikasi)

Indirect memory access dengan operand -1 Operand level pada instruksi.

Selain instruksi dasar stack machine seperti LIT, LOD, STO, JMP, JPC, CAL, dan RET, implementasi kami menambahkan beberapa konvensi internal. Instruksi INT digunakan untuk alokasi stack frame, sedangkan OPR digunakan sebagai dispatcher operasi aritmatika, relasional, boolean, input, dan output. Selain itu, string literal dikodekan menggunakan LIT 0 -1 dengan informasi string pada komentar instruksi, dan akses array dinamis direpresentasikan menggunakan LOD/STO dengan operand -1. Interpreter juga menyediakan HALT sebagai instruksi penghentian eksplisit, walaupun akhir program normal saat ini menggunakan RET 0 0.

BAB 3 PENGUJIAN

1. Test Case 1

input1.txt	
Decorated AST:	
Program(CaseArithmeticDemo) [type=Void, tab=44, block=0, level=0]	
└─ Declarations [type=Unknown, tab=-1, block=0, level=0]	
└─ VarDecl(code) [type=Integer, tab=45, block=0, level=0]	
└─ Type(integer) [type=Integer, tab=33, block=0, level=0]	
└─ VarDecl(base) [type=Integer, tab=46, block=0, level=0]	
└─ Type(integer) [type=Integer, tab=33, block=0, level=0]	
└─ VarDecl(score) [type=Integer, tab=47, block=0, level=0]	
└─ Type(integer) [type=Integer, tab=33, block=0, level=0]	
└─ Block [type=Unknown, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(code) [type=Integer, tab=45, block=0, level=0]	
└─ Literal(2) [type=Integer, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(base) [type=Integer, tab=46, block=0, level=0]	
└─ Literal(5) [type=Integer, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ BinOp(*) [type=Integer, tab=-1, block=0, level=0]	
└─ Var(base) [type=Integer, tab=46, block=0, level=0]	
└─ Literal(2) [type=Integer, tab=-1, block=0, level=0]	
└─ Case [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(code) [type=Integer, tab=45, block=0, level=0]	
└─ Case [type=Unknown, tab=-1, block=0, level=0]	
└─ Literal(1) [type=Integer, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ Literal(10) [type=Integer, tab=-1, block=0, level=0]	
└─ Case [type=Unknown, tab=-1, block=0, level=0]	
└─ Literal(2) [type=Integer, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ Literal(20) [type=Integer, tab=-1, block=0, level=0]	
└─ Case [type=Unknown, tab=-1, block=0, level=0]	
└─ Literal(3) [type=Integer, tab=-1, block=0, level=0]	
└─ Assign [type=Unknown, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ Literal(30) [type=Integer, tab=-1, block=0, level=0]	
└─ ProcedureCall(writeLn) [type=Unknown, tab=40, block=0, level=0]	
└─ Literal(Score =) [type=String, tab=-1, block=0, level=0]	
└─ Var(score) [type=Integer, tab=47, block=0, level=0]	
└─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]	

Symbol Table - tab:

idx	identifrier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	CaseArithmeticDemoProgram		Void	0	1	0	0	43	no
45	code	Variable	Integer	0	1	0	0	44	no
46	base	Variable	Integer	0	1	0	1	45	no
47	score	Variable	Integer	0	1	0	2	46	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	47	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

output1.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```

0 INT 0 6 ; alokasi frame program
1 LIT 0 2 ; literal int 2
2 STO 0 3 ; simpan ke code
3 LIT 0 5 ; literal int 5
4 STO 0 4 ; simpan ke base
5 LOD 0 4 ; load base
6 LIT 0 2 ; literal int 2
7 OPR 0 4 ; op *
8 STO 0 5 ; simpan ke score
9 LOD 0 3 ; load code
10 LIT 0 1 ; literal int 1
11 OPR 0 7 ; case: selector = label

```

```

12 JPC 0 18 ; case: next label
13 LOD 0 5 ; load score
14 LIT 0 10 ; literal int 10
15 OPR 0 2 ; op +
16 STO 0 5 ; simpan ke score
17 JMP 0 36 ; case: end
18 LOD 0 3 ; load code
19 LIT 0 2 ; literal int 2
20 OPR 0 7 ; case: selector = label
21 JPC 0 27 ; case: next label
22 LOD 0 5 ; load score
23 LIT 0 20 ; literal int 20
24 OPR 0 2 ; op +
25 STO 0 5 ; simpan ke score
26 JMP 0 36 ; case: end
27 LOD 0 3 ; load code
28 LIT 0 3 ; literal int 3
29 OPR 0 7 ; case: selector = label
30 JPC 0 36 ; case: next label
31 LOD 0 5 ; load score
32 LIT 0 30 ; literal int 30
33 OPR 0 2 ; op +
34 STO 0 5 ; simpan ke score
35 JMP 0 36 ; case: end
36 LIT 0 -1 ; string:Score =
37 OPR 0 13 ; write arg 0
38 LOD 0 5 ; load score
39 OPR 0 14 ; writeln arg 1
40 RET 0 0 ; akhir program

```

Program Output:

Score = 30

Runtime Status: SUCCESS

2. Test Case 2

input2.txt

Decorated AST:

```

Program(BranchLoopDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(i) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── VarDecl(total) [type=Integer, tab=46, block=0, level=0]
│       └── Type(integer) [type=Integer, tab=33, block=0, level=0]

```

```

└─ Block [type=Unknown, tab=-1, block=0, level=0]
  └─ Assign [type=Unknown, tab=-1, block=0, level=0]
    └─ Var(i) [type=Integer, tab=45, block=0, level=0]
      └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
  └─ Assign [type=Unknown, tab=-1, block=0, level=0]
    └─ Var(total) [type=Integer, tab=46, block=0, level=0]
      └─ Literal(0) [type=Integer, tab=-1, block=0, level=0]
  └─ While [type=Unknown, tab=-1, block=0, level=0]
    └─ BinOp(≤) [type=Boolean, tab=-1, block=0, level=0]
      └─ Var(i) [type=Integer, tab=45, block=0, level=0]
        └─ Literal(5) [type=Integer, tab=-1, block=0, level=0]
    └─ Block [type=Unknown, tab=-1, block=0, level=0]
      └─ If [type=Unknown, tab=-1, block=0, level=0]
        └─ BinOp(>) [type=Boolean, tab=-1, block=0, level=0]
          └─ Var(i) [type=Integer, tab=45, block=0, level=0]
            └─ Literal(2) [type=Integer, tab=-1, block=0, level=0]
          └─ Assign [type=Unknown, tab=-1, block=0, level=0]
            └─ Var(total) [type=Integer, tab=46, block=0, level=0]
              └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
                └─ Var(total) [type=Integer, tab=46, block=0, level=0]
                  └─ Var(i) [type=Integer, tab=45, block=0, level=0]
                └─ Assign [type=Unknown, tab=-1, block=0, level=0]
                  └─ Var(total) [type=Integer, tab=46, block=0, level=0]
                    └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
                      └─ Var(total) [type=Integer, tab=46, block=0, level=0]
                        └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
          └─ Assign [type=Unknown, tab=-1, block=0, level=0]
            └─ Var(i) [type=Integer, tab=45, block=0, level=0]
              └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
                └─ Var(i) [type=Integer, tab=45, block=0, level=0]
                  └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
            └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
        └─ ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
          └─ Literal(Total = ) [type=String, tab=-1, block=0, level=0]
          └─ Var(total) [type=Integer, tab=46, block=0, level=0]
        └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifiser	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	BranchLoopDemo	Program	Void	0	1	0	0	43	no
45	i	Variable	Integer	0	1	0	0	44	no
46	total	Variable	Integer	0	1	0	1	45	no

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	46	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output2.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```

0 INT 0 5 ; alokasi frame program
1 LIT 0 1 ; literal int 1
2 STO 0 3 ; simpan ke i
3 LIT 0 0 ; literal int 0
4 STO 0 4 ; simpan ke total
5 LOD 0 3 ; load i
6 LIT 0 5 ; literal int 5
7 OPR 0 12 ; op ≤
8 JPC 0 27 ; jpc ke akhir while
9 LOD 0 3 ; load i
10 LIT 0 2 ; literal int 2
11 OPR 0 11 ; op >
12 JPC 0 18 ; jpc ke else/end
13 LOD 0 4 ; load total
14 LOD 0 3 ; load i
15 OPR 0 2 ; op +
16 STO 0 4 ; simpan ke total
17 JMP 0 22 ; jmp ke end if
18 LOD 0 4 ; load total
19 LIT 0 1 ; literal int 1
20 OPR 0 2 ; op +
21 STO 0 4 ; simpan ke total
22 LOD 0 3 ; load i
23 LIT 0 1 ; literal int 1
24 OPR 0 2 ; op +
25 STO 0 3 ; simpan ke i
26 JMP 0 5 ; jmp ke awal while
27 LIT 0 -1 ; string:Total =
28 OPR 0 13 ; write arg 0
29 LOD 0 4 ; load total
30 OPR 0 14 ; writeln arg 1

```



```
31 RET 0 0 ; akhir program
```

Program Output:

Total = 14

Runtime Status: SUCCESS

3. Test Case 3

input3.txt

Decorated AST:

```
Program(RepeatForDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(i) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(total) [type=Integer, tab=46, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── Block [type=Unknown, tab=-1, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── Var(i) [type=Integer, tab=45, block=0, level=0]
│       │   └── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── Var(total) [type=Integer, tab=46, block=0, level=0]
│       │   └── Literal(0) [type=Integer, tab=-1, block=0, level=0]
│       └── Repeat [type=Unknown, tab=-1, block=0, level=0]
│           ├── Block [type=Unknown, tab=-1, block=0, level=0]
│           │   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│           │   │   ├── Var(total) [type=Integer, tab=46, block=0, level=0]
│           │   │   └── BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│           │   │       ├── Var(total) [type=Integer, tab=46, block=0, level=0]
│           │   │       └── Var(i) [type=Integer, tab=45, block=0, level=0]
│           │   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│           │   │   ├── Var(i) [type=Integer, tab=45, block=0, level=0]
│           │   │   └── BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│           │   │       ├── Var(i) [type=Integer, tab=45, block=0, level=0]
│           │   │       └── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│           │   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
│           ├── BinOp(>) [type=Boolean, tab=-1, block=0, level=0]
│           │   ├── Var(i) [type=Integer, tab=45, block=0, level=0]
│           │   └── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│           └── For(i, direction=to) [type=Unknown, tab=45, block=0, level=0]
│               ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│               ├── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│               └── Block [type=Unknown, tab=-1, block=0, level=0]
│                   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│                   │   ├── Var(total) [type=Integer, tab=46, block=0, level=0]
│                   │   └── BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│                   │       ├── Var(total) [type=Integer, tab=46, block=0, level=0]
│                   │       └── Var(i) [type=Integer, tab=45, block=0, level=0]
│                   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
└── For(i, direction=down to) [type=Unknown, tab=45, block=0, level=0]
    ├── Literal(3) [type=Integer, tab=-1, block=0, level=0]
```

```

└─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
   └─ Block [type=Unknown, tab=-1, block=0, level=0]
      └─ Assign [type=Unknown, tab=-1, block=0, level=0]
         └─ Var(total) [type=Integer, tab=46, block=0, level=0]
            └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
               └─ Var(total) [type=Integer, tab=46, block=0, level=0]
                  └─ Var(i) [type=Integer, tab=45, block=0, level=0]
                     └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
└─ ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
   └─ Literal(Total = ) [type=String, tab=-1, block=0, level=0]
      └─ Var(total) [type=Integer, tab=46, block=0, level=0]
         └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init

33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	RepeatForDemo	Program	Void	0	1	0	0	43	no
45	i	Variable	Integer	0	1	0	0	44	no
46	total	Variable	Integer	0	1	0	1	45	no

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	46	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output3.txt

Input Mode: DECORATED AST
 Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```

0 INT 0 5 ; alokasi frame program
1 LIT 0 1 ; literal int 1
2 STO 0 3 ; simpan ke i
3 LIT 0 0 ; literal int 0

```

```

4 STO 0 4 ; simpan ke total
5 LOD 0 4 ; load total
6 LOD 0 3 ; load i
7 OPR 0 2 ; op +
8 STO 0 4 ; simpan ke total
9 LOD 0 3 ; load i
10 LIT 0 1 ; literal int 1
11 OPR 0 2 ; op +
12 STO 0 3 ; simpan ke i
13 LOD 0 3 ; load i
14 LIT 0 3 ; literal int 3
15 OPR 0 11 ; op >
16 JPC 0 5 ; jpc ke awal repeat jika belum selesai
17 LIT 0 1 ; literal int 1
18 STO 0 3 ; for: init i
19 LOD 0 3 ; for: load i
20 LIT 0 3 ; literal int 3
21 OPR 0 12 ; for to: cek ≤ limit
22 JPC 0 32 ; jpc ke akhir for
23 LOD 0 4 ; load total
24 LOD 0 3 ; load i
25 OPR 0 2 ; op +
26 STO 0 4 ; simpan ke total
27 LOD 0 3 ; for: load i for step
28 LIT 0 1 ; for: step 1
29 OPR 0 2 ; for to: increment
30 STO 0 3 ; for: update i
31 JMP 0 19 ; jmp ke awal for
32 LIT 0 3 ; literal int 3
33 STO 0 3 ; for: init i
34 LOD 0 3 ; for: load i
35 LIT 0 1 ; literal int 1
36 OPR 0 10 ; for downto: cek ≥ limit
37 JPC 0 47 ; jpc ke akhir for
38 LOD 0 4 ; load total
39 LOD 0 3 ; load i
40 OPR 0 2 ; op +
41 STO 0 4 ; simpan ke total
42 LOD 0 3 ; for: load i for step
43 LIT 0 1 ; for: step 1
44 OPR 0 3 ; for downto: decrement
45 STO 0 3 ; for: update i
46 JMP 0 34 ; jmp ke awal for
47 LIT 0 -1 ; string:Total =
48 OPR 0 13 ; write arg 0
49 LOD 0 4 ; load total

```

```

50 OPR 0 14 ; writeln arg 1
51 RET 0 0 ; akhir program

```

Program Output:

Total = 18

Runtime Status: SUCCESS

4. Test Case 4

input4.txt

Decorated AST:

```

Program(CompositeDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── TypeDecl(Point) [type=Record, tab=47, block=0, level=0]
│   │   ├── RecordType [type=Record, tab=-1, block=5, level=1]
│   │   │   ├── FieldDecl(x) [type=Integer, tab=45, block=5, level=1]
│   │   │   │   ├── Type(integer) [type=Integer, tab=33, block=5, level=1]
│   │   │   │   └── FieldDecl(y) [type=Integer, tab=46, block=5, level=1]
│   │   │   │       ├── Type(integer) [type=Integer, tab=33, block=5, level=1]
│   │   └── VarDecl(p) [type=Record, tab=48, block=0, level=0]
│   │   │   ├── Type(Point) [type=Record, tab=47, block=0, level=0]
│   │   └── VarDecl(values) [type=Array, tab=49, block=0, level=0]
│   │   │   ├── ArrayType [type=Array, tab=-1, block=0, level=0]
│   │   │   │   ├── RangeType [type=Subrange, tab=-1, block=0, level=0]
│   │   │   │   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   │   │   │   │   ├── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│   │   │   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── Block [type=Unknown, tab=-1, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── FieldAccess(x) [type=Integer, tab=45, block=0, level=0]
│       │   │   ├── Var(p) [type=Record, tab=48, block=0, level=0]
│       │   │   └── Literal(4) [type=Integer, tab=-1, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── FieldAccess(y) [type=Integer, tab=46, block=0, level=0]
│       │   │   ├── Var(p) [type=Record, tab=48, block=0, level=0]
│       │   │   └── Literal(6) [type=Integer, tab=-1, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│       │   │   ├── Var(values) [type=Array, tab=49, block=0, level=0]
│       │   │   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       │   │   └── FieldAccess(x) [type=Integer, tab=45, block=0, level=0]
│       │   │       ├── Var(p) [type=Record, tab=48, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│       │   │   ├── Var(values) [type=Array, tab=49, block=0, level=0]
│       │   │   │   ├── Literal(2) [type=Integer, tab=-1, block=0, level=0]
│       │   │   └── FieldAccess(y) [type=Integer, tab=46, block=0, level=0]
│       │   │       ├── Var(p) [type=Record, tab=48, block=0, level=0]
│       ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│       │   ├── ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│       │   │   └── Var(values) [type=Array, tab=49, block=0, level=0]

```

```

└─ Literal(3) [type=Integer, tab=-1, block=0, level=0]
└─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
    └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
        └─ Var(values) [type=Array, tab=49, block=0, level=0]
            └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
        └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
            └─ Var(values) [type=Array, tab=49, block=0, level=0]
                └─ Literal(2) [type=Integer, tab=-1, block=0, level=0]
    └─ ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
        └─ Literal(Sum = ) [type=String, tab=-1, block=0, level=0]
        └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
            └─ Var(values) [type=Array, tab=49, block=0, level=0]
                └─ Literal(3) [type=Integer, tab=-1, block=0, level=0]
└─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identfier	obj	type	ref	nrm	lev	adr	link	init

33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	CompositeDemo	Program	Void	0	1	0	0	43	no
45	x	Field	Integer	0	1	1	0	0	no
46	y	Field	Integer	0	1	1	1	45	no
47	Point	Type	Record[ref=5]0		1	0	0	44	no
48	p	Variable	Record[ref=5]0		1	0	0	47	no
49	values	Variable	Array[ref=1]0		1	0	2	48	no

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	49	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0
5	46	0	0	2

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

1	Subrange	Integer	0	1	3	1	3

output4.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```
0 INT 0 8 ; alokasi frame program
1 LIT 0 4 ; literal int 4
2 STO 0 3 ; simpan ke x
3 LIT 0 6 ; literal int 6
4 STO 0 4 ; simpan ke y
5 LOD 0 3 ; load p.x
6 LIT 0 5 ; array base addr (LValue)
7 LIT 0 1 ; literal int 1
8 LIT 0 1 ; array low (LValue)
9 OPR 0 3 ; idx-low (LValue)
10 OPR 0 2 ; abs addr (LValue)
11 STO 0 -1 ; simpan ke var
12 LOD 0 4 ; load p.y
13 LIT 0 5 ; array base addr (LValue)
14 LIT 0 2 ; literal int 2
15 LIT 0 1 ; array low (LValue)
16 OPR 0 3 ; idx-low (LValue)
17 OPR 0 2 ; abs addr (LValue)
18 STO 0 -1 ; simpan ke var
19 LIT 0 5 ; array base addr
20 LIT 0 1 ; literal int 1
21 LIT 0 1 ; array low bound
22 OPR 0 3 ; index - low
23 OPR 0 2 ; array abs addr
24 LOD 0 -1 ; load array[idx]
25 LIT 0 5 ; array base addr
26 LIT 0 2 ; literal int 2
27 LIT 0 1 ; array low bound
28 OPR 0 3 ; index - low
29 OPR 0 2 ; array abs addr
30 LOD 0 -1 ; load array[idx]
31 OPR 0 2 ; op +
32 LIT 0 5 ; array base addr (LValue)
33 LIT 0 3 ; literal int 3
34 LIT 0 1 ; array low (LValue)
35 OPR 0 3 ; idx-low (LValue)
36 OPR 0 2 ; abs addr (LValue)
37 STO 0 -1 ; simpan ke var
38 LIT 0 -1 ; string:Sum =
39 OPR 0 13 ; write arg 0
40 LIT 0 5 ; array base addr
41 LIT 0 3 ; literal int 3
42 LIT 0 1 ; array low bound
43 OPR 0 3 ; index - low
```

```

44 OPR 0 2 ; array abs addr
45 LOD 0 -1 ; load array[idx]
46 OPR 0 14 ; writeln arg 1
47 RET 0 0 ; akhir program

```

Program Output:
Sum = 10

Runtime Status: SUCCESS

5. Test Case 5

input5.txt

Decorated AST:

```

Program(SubprogramDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(result) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── ProcedureDecl(ShowValue) [type=Procedure, tab=46, block=5, level=1]
│   │   ├── Declarations(parameters) [type=Unknown, tab=-1, block=5, level=1]
│   │   │   ├── Param(x) [type=Integer, tab=47, block=5, level=1]
│   │   │   └── Type(integer) [type=Integer, tab=33, block=5, level=1]
│   │   └── Block [type=Unknown, tab=-1, block=5, level=1]
│   │       ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=5, level=1]
│   │       │   ├── Literal(Value = ) [type=String, tab=-1, block=5, level=1]
│   │       │   └── Var(x) [type=Integer, tab=47, block=5, level=1]
│   │       └── EmptyStatement [type=Unknown, tab=-1, block=5, level=1]
│   └── FunctionDecl(DoubleValue) [type=Integer, tab=48, block=6, level=1]
│       ├── Declarations(parameters) [type=Unknown, tab=-1, block=6, level=1]
│       │   ├── Param(x) [type=Integer, tab=49, block=6, level=1]
│       │   └── Type(integer) [type=Integer, tab=33, block=6, level=1]
│       ├── Block [type=Unknown, tab=-1, block=6, level=1]
│       │   ├── Assign [type=Unknown, tab=-1, block=6, level=1]
│       │   │   ├── Var(DoubleValue) [type=Integer, tab=48, block=6, level=1]
│       │   │   └── BinOp(*) [type=Integer, tab=-1, block=6, level=1]
│       │   │       ├── Var(x) [type=Integer, tab=49, block=6, level=1]
│       │   │       └── Literal(2) [type=Integer, tab=-1, block=6, level=1]
│       │   └── EmptyStatement [type=Unknown, tab=-1, block=6, level=1]
│       └── Type(integer) [type=Integer, tab=33, block=0, level=0]
├── Block [type=Unknown, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(result) [type=Integer, tab=45, block=0, level=0]
│   │   └── FunctionCall(DoubleValue) [type=Integer, tab=48, block=0, level=0]
│   │       └── Literal(7) [type=Integer, tab=-1, block=0, level=0]
│   ├── ProcedureCall(ShowValue) [type=Unknown, tab=46, block=0, level=0]
│   │   └── Var(result) [type=Integer, tab=45, block=0, level=0]
│   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init

33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	SubprogramDemo	Program	Void	0	1	0	0	43	no
45	result	Variable	Integer	0	1	0	0	44	no
46	ShowValue	Procedure	Procedure	5	1	0	0	45	no
47	x	Parameter	Integer	0	1	1	0	0	no
48	DoubleValue	Function	Integer	6	1	0	1	46	no
49	x	Parameter	Integer	0	1	1	0	0	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	48	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0
5	47	0	1	1
6	49	0	1	1

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output5.txt	
Input Mode: DECORATED AST	
Semantic Analysis: SKIPPED (Decorated AST input)	
Intermediate Code:	
0 INT 0 4 ; alokasi frame program	
1 JMP 0 8 ; jmp skip body ShowValue	
2 INT 1 5 ; alokasi frame ShowValue	
3 LIT 0 -1 ; string:Value =	
4 OPR 0 13 ; write arg 0	
5 LOD 0 3 ; load x	
6 OPR 0 14 ; writeln arg 1	
7 RET 0 0 ; ret dari ShowValue	
8 JMP 0 15 ; jmp skip body DoubleValue	
9 INT 1 5 ; alokasi frame DoubleValue	
10 LOD 0 3 ; load x	
11 LIT 0 2 ; literal int 2	


```

12 OPR 0 4 ; op *
13 STO 1 4 ; simpan ke DoubleValue
14 RET 0 0 ; ret dari DoubleValue
15 LIT 0 7 ; literal int 7
16 CAL 0 9 ; call DoubleValue
17 LOD 0 4 ; load return value DoubleValue
18 STO 0 3 ; simpan ke result
19 LOD 0 3 ; load result
20 CAL 0 2 ; call ShowValue
21 RET 0 0 ; akhir program

```

Program Output:

Value = 14

Runtime Status: SUCCESS

6. Test Case 6

input6.txt

Decorated AST:

```

Program(RuntimeDivZeroDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(x) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── VarDecl(y) [type=Integer, tab=46, block=0, level=0]
│       └── Type(integer) [type=Integer, tab=33, block=0, level=0]
├── Block [type=Unknown, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(x) [type=Integer, tab=45, block=0, level=0]
│   │   └── Literal(10) [type=Integer, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(y) [type=Integer, tab=46, block=0, level=0]
│   │   └── Literal(0) [type=Integer, tab=-1, block=0, level=0]
│   ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
│   │   └── Literal(Before division) [type=String, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(x) [type=Integer, tab=45, block=0, level=0]
│   │   └── BinOp(div) [type=Integer, tab=-1, block=0, level=0]
│   │       ├── Var(x) [type=Integer, tab=45, block=0, level=0]
│   │       └── Var(y) [type=Integer, tab=46, block=0, level=0]
│   ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
│   │   ├── Literal(After division = ) [type=String, tab=-1, block=0, level=0]
│   │   └── Var(x) [type=Integer, tab=45, block=0, level=0]
│   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no

35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	RuntimeDivZeroDemoProgram		Void	0	1	0	0	43	no
45	x	Variable	Integer	0	1	0	0	44	no
46	y	Variable	Integer	0	1	0	1	45	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	46	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output6.txt	
Input Mode: DECORATED AST	
Semantic Analysis: SKIPPED (Decorated AST input)	
Intermediate Code:	
0 INT 0 5 ; alokasi frame program	
1 LIT 0 10 ; literal int 10	
2 STO 0 3 ; simpan ke x	
3 LIT 0 0 ; literal int 0	
4 STO 0 4 ; simpan ke y	
5 LIT 0 -1 ; string:Before division	
6 OPR 0 14 ; writeln arg 0	
7 LOD 0 3 ; load x	
8 LOD 0 4 ; load y	
9 OPR 0 5 ; op div	
10 STO 0 3 ; simpan ke x	
11 LIT 0 -1 ; string:After division =	
12 OPR 0 13 ; write arg 0	
13 LOD 0 3 ; load x	
14 OPR 0 14 ; writeln arg 1	
15 RET 0 0 ; akhir program	
Program Output:	

Before division

Runtime Status: FAILED

Runtime Error:

- Division by zero at instruction 9

7. Test Case 7

input7.txt

Decorated AST:

```
Program(RuntimeInvalidMemoryDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── TypeDecl(Numbers) [type=Array, tab=45, block=0, level=0]
│   │   ├── ArrayType [type=Array, tab=-1, block=0, level=0]
│   │   │   ├── RangeType [type=Subrange, tab=-1, block=0, level=0]
│   │   │   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   │   │   │   └── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── VarDecl(values) [type=Array, tab=46, block=0, level=0]
│       ├── Type(Numbers) [type=Array, tab=45, block=0, level=0]
│       └── Block [type=Unknown, tab=-1, block=0, level=0]
│           ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│           │   ├── ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│           │   │   ├── Var(values) [type=Array, tab=46, block=0, level=0]
│           │   │   └── Literal(100000) [type=Integer, tab=-1, block=0, level=0]
│           │   └── Literal(7) [type=Integer, tab=-1, block=0, level=0]
│           ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
│           │   └── Literal(Should not reach here) [type=String, tab=-1, block=0, level=0]
│           └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
```

Symbol Table - tab:

idx	identifrier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	RuntimeInvalidMemoryDemoProgram		Void		0	1	0	0	43 no
45	Numbers	Type	Array[ref=1]0		1	0	0	44	no
46	values	Variable	Array[ref=1]0		1	0	0	45	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	46	0	0	0

1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size
1	Subrange	Integer	0	1	3	1	3

output7.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```
0 INT 0 6 ; alokasi frame program
1 LIT 0 7 ; literal int 7
2 LIT 0 3 ; array base addr (LValue)
3 LIT 0 100000 ; literal int 100000
4 LIT 0 1 ; array low (LValue)
5 OPR 0 3 ; idx-low (LValue)
6 OPR 0 2 ; abs addr (LValue)
7 STO 0 -1 ; simpan ke var
8 LIT 0 -1 ; string:Should not reach here
9 OPR 0 14 ; writeln arg 0
10 RET 0 0 ; akhir program
```

Program Output:

Runtime Status: FAILED

Runtime Error:

- Invalid memory access at address 100002 at instruction 7

8. Test Case 8

input8.txt

Decorated AST:

Program(RuntimeInfiniteLoopDemo) [type=Void, tab=44, block=0, level=0]

```
├─ Declarations [type=Unknown, tab=-1, block=0, level=0]
│   └─ VarDecl(x) [type=Integer, tab=45, block=0, level=0]
│       └─ Type(integer) [type=Integer, tab=33, block=0, level=0]
├─ Block [type=Unknown, tab=-1, block=0, level=0]
│   └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│       ├── Var(x) [type=Integer, tab=45, block=0, level=0]
│       └─ Literal(0) [type=Integer, tab=-1, block=0, level=0]
```

```

└─ While [type=Unknown, tab=-1, block=0, level=0]
  └─ Var(true) [type=Boolean, tab=38, block=0, level=0]
    └─ Block [type=Unknown, tab=-1, block=0, level=0]
      └─ Assign [type=Unknown, tab=-1, block=0, level=0]
        └─ Var(x) [type=Integer, tab=45, block=0, level=0]
          └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
            └─ Var(x) [type=Integer, tab=45, block=0, level=0]
              └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
            └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
          └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
        └─ ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
          └─ Literal(Should not reach here) [type=String, tab=-1, block=0, level=0]
        └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init

33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	RuntimeInfiniteLoopDemoProgram		Void		0	1	0	0	43 no
45	x	Variable	Integer	0	1	0	0	44	no

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	45	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output8.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```

0 INT 0 4 ; alokasi frame program
1 LIT 0 0 ; literal int 0
2 STO 0 3 ; simpan ke x
3 LIT 0 1 ; const true

```

```

4 JPC 0 10 ; jpc ke akhir while
5 LOD 0 3 ; load x
6 LIT 0 1 ; literal int 1
7 OPR 0 2 ; op +
8 STO 0 3 ; simpan ke x
9 JMP 0 3 ; jmp ke awal while
10 LIT 0 -1 ; string:Should not reach here
11 OPR 0 14 ; writeln arg 0
12 RET 0 0 ; akhir program

```

Program Output:

Runtime Status: FAILED

Runtime Error:

- Execution step limit exceeded at instruction 8

9. Test Case 9

input9.txt

Decorated AST:

```

Program(RuntimeStackOverflowDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── ProcedureDecl(Recurse) [type=Procedure, tab=45, block=5, level=1]
│   │   └── Block [type=Unknown, tab=-1, block=5, level=1]
│   │       ├── ProcedureCall(Recurse) [type=Unknown, tab=45, block=5, level=1]
│   │       └── EmptyStatement [type=Unknown, tab=-1, block=5, level=1]
└── Block [type=Unknown, tab=-1, block=0, level=0]
    ├── ProcedureCall(Recurse) [type=Unknown, tab=45, block=0, level=0]
    └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	RuntimeStackOverflowDemoProgram			Void	0	1	0	0	43 no
45	Recurse	Procedure	Procedure	5	1	0	0	44	no

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	45	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0
5	0	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

output9.txt

Input Mode: DECORATED AST

Semantic Analysis: SKIPPED (Decorated AST input)

Intermediate Code:

```
0 INT 0 3 ; alokasi frame program
1 JMP 0 5 ; jmp skip body Recurse
2 INT 1 3 ; alokasi frame Recurse
3 CAL 1 2 ; call Recurse
4 RET 0 0 ; ret dari Recurse
5 CAL 0 2 ; call Recurse
6 RET 0 0 ; akhir program
```

Program Output:

Runtime Status: FAILED

Runtime Error:

- Stack overflow: need 3 slots at instruction 3

BAB 4

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada Milestone 4 pengembangan kompilator bahasa Arion ini, kelompok kami telah berhasil mengimplementasikan tahap Intermediate Code Generation dan Interpreter. Masukan utama pada tahap ini berupa Decorated AST beserta informasi Symbol Table hasil Milestone 3, kemudian diproses menjadi instruksi stack machine yang merepresentasikan langkah-langkah eksekusi program. Dengan demikian, alur kompilasi yang sebelumnya berhenti pada analisis semantik kini telah dilanjutkan hingga tahap eksekusi program.

Implementasi Intermediate Code Generator mampu menerjemahkan berbagai konstruksi bahasa Arion, seperti assignment, ekspresi aritmatika dan boolean, percabangan, perulangan, array, record, procedure, function, serta pemanggilan write dan writeln. Mekanisme pengembalian nilai fungsi juga telah ditangani melalui assignment ke function identifier. Instruksi yang dihasilkan kemudian dieksekusi oleh Interpreter berbasis stack machine dengan dukungan operasi seperti LIT, LOD, STO, OPR, JMP, JPC, CAL, dan RET.

Selain itu, program juga telah dilengkapi dengan mekanisme penanganan runtime error sebagai bagian dari bonus vulnerabilities pada interpreter. Beberapa kondisi seperti division by zero, invalid memory access, infinite loop melalui batas langkah eksekusi, dan stack overflow akibat rekursi tanpa basis telah berhasil dideteksi dan dilaporkan tanpa menyebabkan program crash. Dengan adanya pengujian terhadap kasus sukses maupun kasus gagal saat runtime, implementasi Milestone 4 ini telah memenuhi tujuan utama, yaitu mengubah Decorated AST menjadi Intermediate Code dan mengeksekusinya menggunakan interpreter.

2. Saran

Selama mengerjakan Milestone 4 ini, kami menemukan beberapa hal yang dapat menjadi catatan untuk pengembangan berikutnya. Pertama, dari sisi implementasi, validasi terhadap masukan Decorated AST masih dapat dibuat lebih eksplisit. Saat ini program mengasumsikan bahwa Decorated AST yang diterima merupakan hasil valid dari Milestone 3. Ke depannya, dapat ditambahkan modul khusus untuk memvalidasi konsistensi antara node AST, atribut semantik, tab, btab, dan atab sebelum proses code generation dilakukan. Kedua, sistem pengujian otomatis sangat disarankan untuk terus dikembangkan. Pada tahap ini, pengujian tidak hanya perlu mencakup program yang

berhasil dieksekusi, tetapi juga kasus-kasus runtime error seperti division by zero, invalid memory access, infinite loop, dan stack overflow. Dengan adanya testing yang lebih sistematis, perubahan pada code generator maupun interpreter dapat segera diverifikasi agar tidak merusak fitur yang sudah berjalan. Ketiga, kami menyarankan agar spesifikasi tugas besar pada milestone berikutnya dapat ditulis dengan lebih jelas dan eksplisit, terutama terkait format masukan, format keluaran, serta batasan pengujian yang diharapkan. Pada Milestone 4, terdapat beberapa bagian yang dapat menimbulkan interpretasi berbeda, misalnya apakah masukan berupa source code atau Decorated AST, apakah Symbol Table termasuk bagian dari Decorated AST, serta bagaimana cara ideal menguji bonus interpreter vulnerabilities. Penjelasan yang lebih rinci akan sangat membantu mahasiswa dalam menentukan desain implementasi yang sesuai sejak awal.

LAMPIRAN

Tautan repository Github:

<https://github.com/Rizelbit/SBT-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No.	NIM	Persentase Pembagian Tugas (%)
1.	13524032	25
2.	13524036	25
3.	13524056	25
4.	13524102	25

REFERENSI

“Introduction to Automata Theory, Languages, and Computation 3rd Edition”

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

“Compilers - Principles, Techniques, and Tools (2006)”

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

“Introduction To Automata Theory Languages , and Computation - John Hopcroft.”

https://methodist.edu.in/web/uploads/files/Introduction%20To%20Automata%20Theory_john%20hopcroft.pdf

“Syntax Directed Translation - GeeksforGeeks”

<https://www.geeksforgeeks.org/syntax-directed-translation-in-compiler-design/>